

Building the WhatsApp Ordering Chatbot

A complete, plain-language build guide: how to stand up a WhatsApp chatbot, wire it to Claude, OpenAI, or Gemini, and connect that AI agent to Bharat MCP — so a customer can order food entirely inside a WhatsApp chat.

Customer: "Hi! Can I get a Paneer Butter Masala from Spice Route?"

Bot: "Yes — ₹280, ready in ~25 min. Confirm to order?"

Contents

1 What This Handbook Covers

2 The Cast of Characters

3 The Big Picture — Three Layers, One Journey

PART ONE — THE WHATSAPP LAYER

4 Setting Up the WhatsApp Business Number

5 Receiving and Sending Messages

6 Remembering the Conversation

PART TWO — THE AI AGENT LAYER

7 The Shared Idea Behind All Three AI Providers

8 Connecting to Claude, OpenAI, and Gemini

9 Choosing One Agent, or Routing Across All Three

PART THREE — WIRING IN BHARAT MCP

10 Connecting the AI Agent to Bharat MCP

11 The Full Journey — A WhatsApp Order, Start to Finish

REFERENCE

12 Technical Requirements Checklist

13 Business & Compliance Checklist

14 Risks & Limitations

15 Glossary of Terms

16 Frequently Asked Questions

1 What This Handbook Covers

The first Bharat MCP handbook explained what happens once an AI agent is already talking to Bharat MCP. The second explained how Bharat MCP shows up as a connector inside Claude.ai itself. This third handbook covers a different path entirely — the one where **Reward360 builds its own front end**, and a WhatsApp chat is that front end.

THINK OF IT THIS WAY

In the second handbook, the AI concierge lived inside someone else's building (Claude.ai) and Reward360 simply asked to be listed as a service the concierge could call on. In this handbook, Reward360 builds and staffs **its own concierge desk** — reachable through a WhatsApp number instead of a lobby — and hires an AI model (Claude, OpenAI, or Gemini) as the concierge working behind that desk. The customer never sees the AI model by name, never signs into anything, and never leaves WhatsApp. They just chat, and an order happens.

Three things have to be built and connected for this to work, and this handbook covers each one, in order:

1. **The WhatsApp layer** — a number customers message, and a server that can receive and send WhatsApp messages
2. **The AI agent layer** — that server calling Claude, OpenAI, or Gemini to actually understand and respond to what the customer said
3. **The MCP layer** — the AI agent, in turn, calling Bharat MCP to check menus, availability, and place real orders

2 The Cast of Characters

Several of these are new, specific to this path. A few are familiar from the earlier handbooks.

CHARACTER	WHAT IT ACTUALLY IS	WHAT IT DOES IN THIS STORY
The Customer	Anyone with WhatsApp installed	Sends a normal chat message — no app to download, no account to create
WhatsApp Business Platform	Meta's system for businesses to send/receive WhatsApp messages programmatically	Carries every message between the customer's phone and Reward360's server
Reward360's Backend Server	A server Reward360 builds and runs — the new piece in this handbook	Receives each WhatsApp message, decides what to do, and sends a reply back
The AI Agent (Claude / OpenAI / Gemini)	A large language model, reached through its maker's API	Reads the conversation, understands intent, and decides which Bharat MCP tool to call
Bharat MCP Server	The same server from the first handbook	Answers the AI agent's tool calls — menus, availability, orders

ONE IMPORTANT DIFFERENCE FROM THE CONNECTOR PATH

In the second handbook, the customer had to explicitly "connect" Bharat MCP inside Claude.ai. Here, there is no such step for the customer at all — Reward360's own backend is the one holding the connection to the AI provider and to Bharat MCP. The customer experiences none of that; they just see a WhatsApp chat that happens to be very good at finding and ordering food.

3 The Big Picture — Three Layers, One Journey

Here is the complete map. Notice it has exactly three "layers" — WhatsApp, the AI agent, and Bharat MCP — with Reward360's own backend server sitting in the middle, holding the whole thing together.

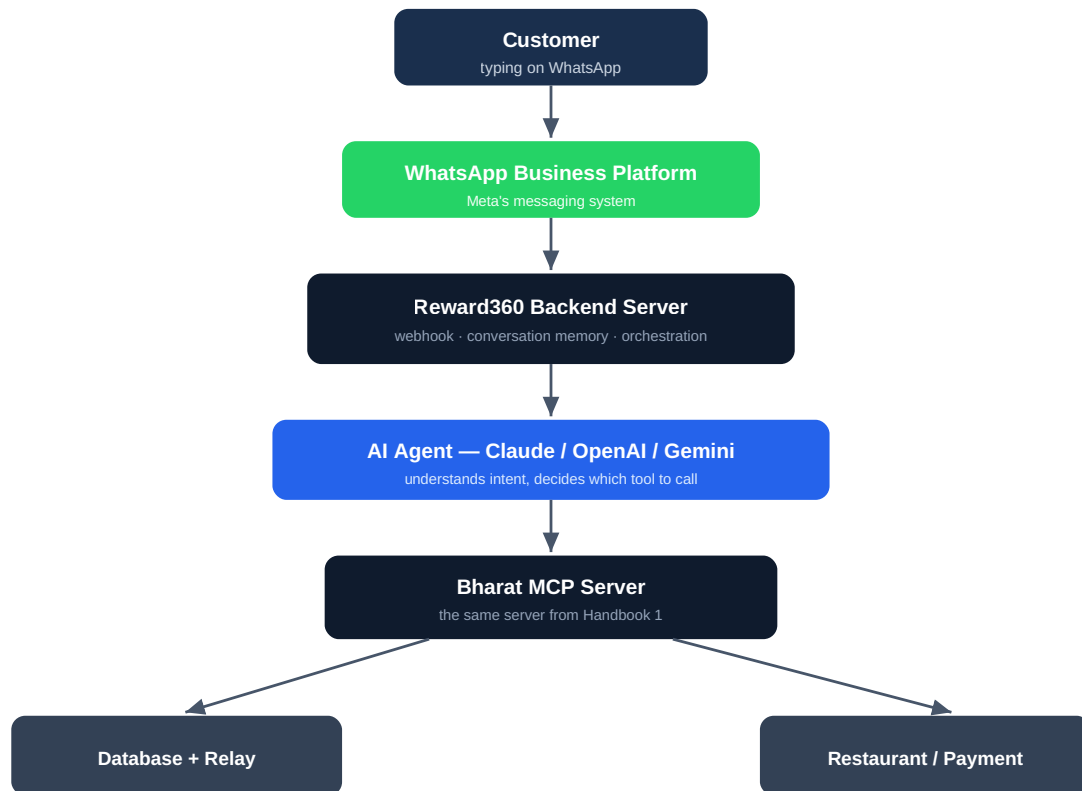


Fig 1. Five layers, top to bottom. Reward360 owns and operates every box except the customer's phone and WhatsApp's own platform.

Why the backend server is the important new piece

In the first two handbooks, "Claude" was the thing directly facing the customer. Here, **Reward360's own backend server** is the thing facing the customer (through WhatsApp), and it's the one deciding, message by message, to call an AI model and then relay that model's decisions onward to Bharat MCP. This server is new work — it didn't exist in either of the earlier two paths — and Parts One through Three of this handbook are about building exactly this server, piece by piece.

4 Setting Up the WhatsApp Business Number

Before any code is written, Reward360 needs an actual WhatsApp number that can send and receive messages programmatically — not a regular personal WhatsApp account.

1 Choose direct-with-Meta, or a Business Solution Provider (BSP)

Reward360 can register directly with Meta's WhatsApp Cloud API, or go through a BSP such as Gupshup, Twilio, or MessageBird, which sits between Reward360 and Meta. A BSP costs a bit more per message but gives a dashboard, easier onboarding, and support — this exact decision is already flagged as an open question in Bharat MCP's own Non-Technical Requirements document.

2 Complete Meta Business verification

Meta requires proof that Reward360 is a real, registered business before issuing a WhatsApp Business number capable of messaging at scale. This includes business documents and a display name that goes through its own approval.

3 Register and verify a phone number

This becomes the number customers message — it can be a new number or an existing one, but once registered for the WhatsApp Business Platform, it can no longer be used in the regular WhatsApp consumer app.

4 Create and get message templates approved

WhatsApp requires businesses to use pre-approved **message templates** for any message that isn't a direct reply within an active conversation — for example, a proactive "your order is ready" nudge sent after a gap. Templates are submitted to Meta for approval before they can be used.

A RULE WORTH UNDERSTANDING EARLY: THE 24-HOUR WINDOW

WhatsApp only allows a business to send free-form messages within **24 hours** of the customer's last message. Inside that window, the bot can say anything. Outside it, only pre-approved templates can be sent. For food ordering this is rarely an issue — most of the conversation and status updates happen within an hour — but it matters if, say, a restaurant confirms an order two days late, or a customer wants to be re-engaged after going quiet.

5 Receiving and Sending Messages

With a number in place, the next piece is the actual plumbing: code that receives an incoming WhatsApp message and code that sends a reply.

5.1 Receiving messages — the webhook

WHAT IS A WEBHOOK?

A webhook is simply a URL that Reward360's server exposes, which Meta calls automatically the instant a customer sends a message — the reverse of Reward360 having to constantly ask "any new messages?" Meta pushes the message to Reward360 the moment it arrives.

1 Reward360 builds and registers a webhook endpoint

A specific URL (e.g. `https://bharat-mcp.reward360.com/whatsapp/webhook`) is registered with Meta or the chosen BSP as the destination for all incoming message events.

2 Every incoming message triggers a call to that URL

The payload includes the customer's phone number, the message text, and a timestamp — this is the raw material the rest of the system works from.

3 The server verifies the message is genuinely from Meta

Meta signs each webhook call with a secret key. Reward360's server checks this signature before trusting the message — without this check, anyone who discovered the webhook URL could send fake "messages" pretending to be a customer.

5.2 Sending messages back

Replying works the opposite way: Reward360's server calls a separate "send message" API endpoint provided by Meta or the BSP, including the customer's phone number and the reply text. There's no ongoing connection to maintain — each message, in either direction, is its own independent request.

Hi! Can I get a Paneer Butter Masala from Spice Route?

Let me check that for you...

Yes — ₹280, ready in about 25 minutes. Shall I place the order?

Yes please

Order placed! Spice Route has been notified — I'll let you know the moment they confirm.

6 Remembering the Conversation

This is the detail most often missed by people new to building this kind of system, so it earns its own section.

AI MODELS DON'T REMEMBER ANYTHING BY THEMSELVES

Every single call to Claude, OpenAI, or Gemini is completely independent — the model has no memory of the customer's previous message unless Reward360's server explicitly sends the whole conversation, every time. If the customer says "yes" to confirm an order, the AI agent only knows what "yes" refers to if the earlier messages (the menu check, the price, the question "shall I place the order?") are included again in that same request.

1 Every customer phone number gets its own conversation history

Reward360's server stores, per phone number, the running list of messages exchanged so far — this can live in the same database already holding restaurants and orders.

2 Each new message is added to that history before calling the AI agent

When "yes please" arrives, the server doesn't send just those two words — it sends the whole conversation so far, with "yes please" appended as the newest message.

3 The AI agent's reply gets added back into that same history

So the next message the customer sends will again include everything that came before — the conversation accumulates, message by message, entirely on Reward360's own server.

WORTH KNOWING

This is exactly the same underlying idea as a person re-reading an entire email thread before writing a reply, rather than remembering it from memory. It's more data sent with every request than it might seem necessary — but it's what makes a stateless AI model behave like it's having one continuous conversation.

7 The Shared Idea Behind All Three AI Providers

Claude, OpenAI's models, and Gemini are built by three different companies, with three different APIs — but the core idea Reward360's backend relies on is the same across all three, and it's worth understanding once, clearly, before looking at the differences.

THINK OF IT THIS WAY

Imagine handing a very capable assistant a phone and a short list of phone numbers they're allowed to call — "the menu line," "the ordering line," "the status line" — along with a one-line description of what each number is for. You never dial the numbers yourself; you just describe the situation, and the assistant decides which number (if any) makes sense to call, dials it, listens to the answer, and gets back to you. That's what "giving an AI model access to tools" means, whichever company built the model.

In every case, the pattern is the same three-part loop:

1. Reward360's server tells the AI model: here's the conversation so far, and here's the list of tools you're allowed to use (Bharat MCP's `search_restaurants` , `get_menu` , and so on)
2. The AI model decides, on its own, whether calling one of those tools makes sense right now — and if so, which one, with what details filled in
3. Reward360's server actually makes that call (to Bharat MCP), gets the real answer, and gives it back to the AI model to continue reasoning or reply to the customer

What differs between Claude, OpenAI, and Gemini is **the exact shape of the request** used to describe those tools and receive that decision — not the underlying idea.

8 Connecting to Claude, OpenAI, and Gemini

8.1 Claude — direct MCP support

Claude's API has built-in support for connecting an MCP server like Bharat MCP directly into a single request — Reward360's server doesn't need to describe each tool by hand. It simply points Claude at Bharat MCP's address:

```
POST https://api.anthropic.com/v1/messages

{
  "model": "claude-sonnet-4-6",
  "messages": [ ...conversation so far... ],
  "mcp_servers": [
    { "type": "url", "url": "https://bharat-mcp.reward360.com/mcp",
      "name": "bharat-mcp" }
  ]
}
```

Claude reads Bharat MCP's own tool list directly from the server, decides which tool to call, and the API handles much of the back-and-forth automatically.

8.2 OpenAI and Gemini — tool/function calling

Both OpenAI's and Google's Gemini APIs support the same underlying concept — commonly called **function calling** or **tool use** — where Reward360's server describes each Bharat MCP tool (its name, its purpose, and what details it needs) directly in the request, and the model responds by saying "call this tool, with these details" rather than calling it directly itself. Reward360's backend then makes the actual call to Bharat MCP and reports the result back.

CONFIRM BEFORE BUILDING AGAINST THIS

Whether OpenAI's and Gemini's APIs offer a direct, built-in way to point at a remote MCP server the way Claude's `mcp_servers` field does — versus requiring Bharat MCP's tools to be manually described in each request — can change as these platforms evolve quickly. This is exactly the kind of detail to verify directly against OpenAI's and Google's own current API documentation before finalizing the backend's design, rather than assuming it matches what's described here.

8.3 The practical difference for Reward360's backend

PROVIDER	WHAT REWARD360'S BACKEND NEEDS TO DO
Claude	Point directly at Bharat MCP's address in the request; Claude handles tool discovery itself

OpenAI	Describe each Bharat MCP tool's name and required details in the request; relay the model's requested tool call to Bharat MCP manually
Gemini	Same pattern as OpenAI — describe the tools, relay the requested call manually

9 Choosing One Agent, or Routing Across All Three

Reward360's backend doesn't have to commit to only one AI provider. Three realistic approaches:

APPROACH	HOW IT WORKS	WHEN IT MAKES SENSE
Single provider	Every conversation goes to one chosen model (e.g., always Claude)	Simplest to build and test — the right starting point for a pilot
Fallback	A primary provider is used; if its API is slow or down, the backend automatically retries with a second provider	Adds resilience once the pilot is stable and reliability matters more
A/B or split routing	Different conversations are deliberately sent to different providers, and results are compared	Useful later, to compare cost, response quality, or speed across providers — not a pilot-stage priority

For a first build, a single provider is the right choice — the complexity of routing or fallback logic isn't worth adding until the core WhatsApp-to-order flow is proven end to end with just one.

10 Connecting the AI Agent to Bharat MCP

This is the same territory covered in detail in the first Bharat MCP handbook — the six tools, the order lifecycle, the failure handling — so it isn't repeated in full here. What's new in this handbook is *where* that connection is configured: inside Reward360's own backend server's code, as part of every request it sends to the AI provider, rather than inside Claude.ai's own settings (as it was in the connector handbook).

WORTH RESTATING CLEARLY

Whichever AI provider is used, Bharat MCP itself does not change. It's the exact same server, exposing the exact same six tools (`search_restaurants` , `get_menu` , `check_availability` , `place_order` , `get_order_status` , `book_table`), regardless of whether it's being called by Claude directly, or by OpenAI/Gemini through Reward360's backend relaying the call. This is the entire point of building Bharat MCP as a standard MCP server in the first place — one system, usable from anywhere.

The one addition specific to this path: since there's no individual end-user sign-in step (unlike the connector path), Bharat MCP can authenticate the backend server itself using a single, securely stored API key or token — proving that calls are genuinely coming from Reward360's own trusted server, not from an outside party.

11 The Full Journey — A WhatsApp Order, Start to Finish

Now the complete picture, tracing the same example used throughout the Bharat MCP documents — checking and ordering a Paneer Butter Masala — but this time starting and ending inside WhatsApp.

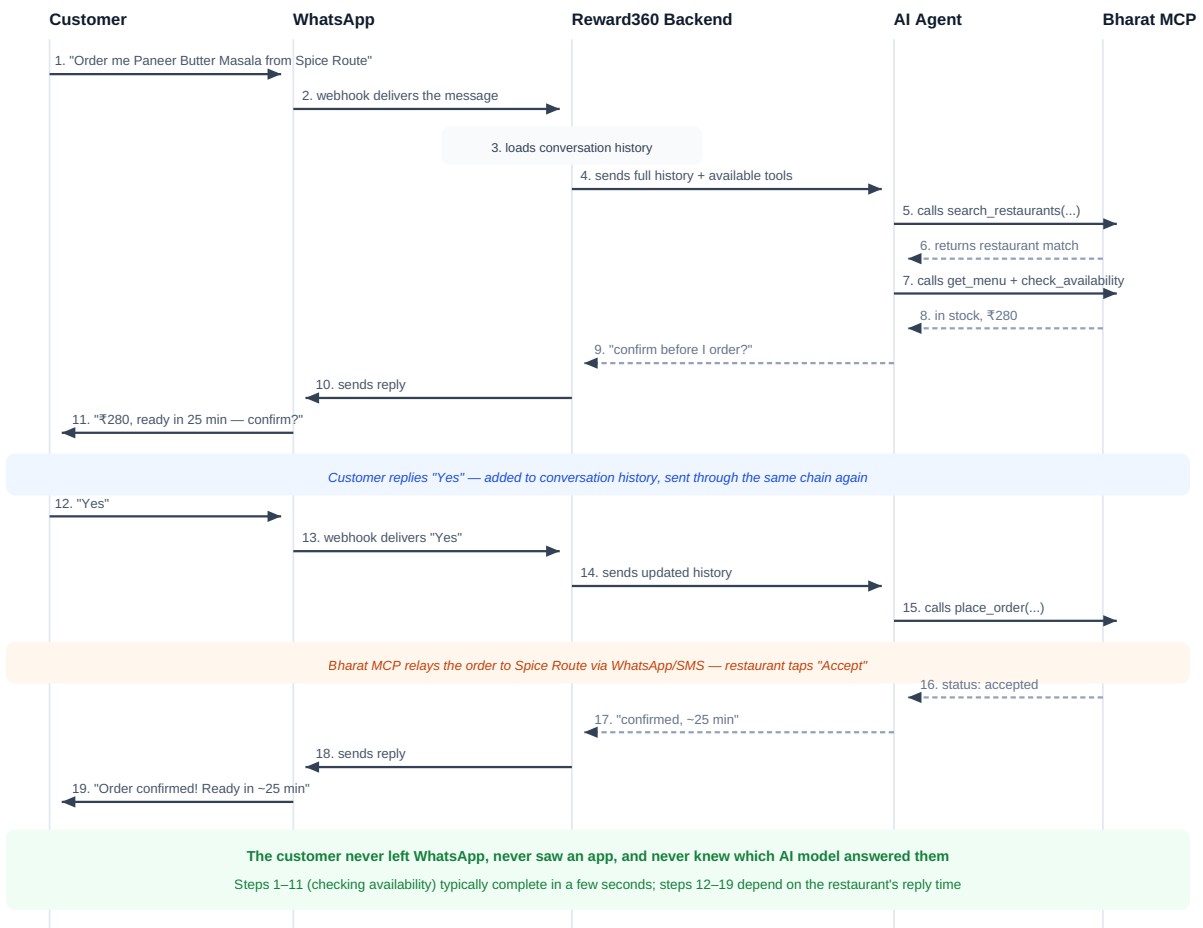


Fig 2. Every one of the six Bharat MCP tools from Handbook 1 shows up here — just reached through Reward360's own backend instead of directly.

12 Technical Requirements Checklist

Everything in this table is new work specific to the WhatsApp chatbot path — additional to what the core Bharat MCP infra plan and FRD already specify.

REQUIREMENT	WHY IT'S NEEDED
WhatsApp Business number + API access	The actual channel customers message — via Meta directly or a BSP
A public webhook endpoint	So Meta can deliver incoming messages to Reward360's server the instant they arrive
Webhook signature verification	Confirms incoming messages genuinely come from Meta, not a spoofed source
Message deduplication	Meta may redeliver the same message event; the backend needs to recognize and ignore duplicates, the same idea as the idempotency key on <code>place_order</code>
Per-customer conversation storage	So the AI agent can be given the full message history on every call, since it has no memory of its own
An AI provider account and API key (Claude, OpenAI, and/or Gemini)	Needed to actually call the chosen model(s)
Securely stored Bharat MCP credentials	Authenticates the backend server itself to Bharat MCP, since there's no individual customer sign-in step on this path
Logging and monitoring across all three layers	A failure could originate in WhatsApp delivery, the AI provider's API, or Bharat MCP itself — clear logs are what let a failure be traced back to its actual layer

13 Business & Compliance Checklist

REQUIREMENT	WHY IT'S NEEDED
Meta Business verification	Required before a WhatsApp Business number can message customers at any real scale
Message template approval	Needed for any message sent outside the 24-hour reply window (see Section 4)
Customer opt-in	WhatsApp's own business policy requires customers to have clearly opted in before a business messages them — this needs a defined moment of consent (e.g., the customer messaging the number first, which counts as opt-in)
A privacy disclosure specific to this channel	Customers should be told, in plain language, that an AI system is involved in handling their order — consistent with the data-sharing/privacy points already raised in the Non-Technical Requirements document
Provider terms of service review	Anthropic's, OpenAI's, and Google's API terms should each be reviewed for anything relevant to this specific use case — real payments, personal contact data, a regulated food-ordering context

14 Risks & Limitations

THREE POINTS OF FAILURE INSTEAD OF ONE

A problem can now originate in WhatsApp's delivery, the AI provider's API, or Bharat MCP itself — each needs its own monitoring, and a customer-facing error message needs to make sense regardless of which layer actually failed.

CONVERSATION HISTORY GROWS, AND COSTS FOLLOW

Because the full conversation is resent on every single message (Section 6), a long back-and-forth costs more, in both time and API usage, than a single short exchange. Long conversations may need to be intelligently trimmed or summarized rather than sent in full forever.

THE 24-HOUR RULE CAN STRAND A SLOW ORDER

If a restaurant takes unusually long to respond and the window lapses, sending an unprompted status update requires a pre-approved template rather than a free-form message — this needs to be planned for, not discovered during the pilot.

MULTI-PROVIDER COMPLEXITY IS EASY TO UNDERESTIMATE

Supporting Claude, OpenAI, and Gemini side by side (Section 9) means three separate integrations to build, test, and keep working as each provider changes its own API over time — a single-provider start (Section 9) is the sensible way to avoid this becoming the project's biggest source of ongoing maintenance.

15 Glossary of Terms

Webhook

A URL a server exposes so another system (here, Meta) can push events to it the instant they happen, rather than being repeatedly asked.

WhatsApp Business Platform

Meta's system for businesses to send and receive WhatsApp messages through code rather than the regular consumer app.

Business Solution Provider (BSP)

A company (like Gupshup or Twilio) that sits between a business and Meta, simplifying WhatsApp Business Platform setup in exchange for a per-message fee.

24-Hour Window

The rule that free-form WhatsApp replies are only allowed within 24 hours of the customer's last message; outside that window, only pre-approved templates can be sent.

Message Template

A pre-written, pre-approved message format required for any WhatsApp message sent outside the 24-hour window.

Function Calling / Tool Use

The general capability, shared across Claude, OpenAI, and Gemini, that lets an AI model request that a specific outside action be taken (like checking a menu) rather than only replying in plain text.

Conversation History

The full record of messages exchanged with one customer, stored by Reward360's own server and resent with every new message, since the AI model itself has no memory between requests.

Fallback

A design where a backup AI provider is automatically used if the primary one is unavailable or too slow.

16 Frequently Asked Questions

Q: Does the customer need to know an AI model is involved at all?

They should be told plainly, even briefly, that an automated assistant is handling the conversation — this is both good practice and consistent with the disclosure expectations already raised elsewhere in the Bharat MCP documents.

Q: What happens if the customer sends a voice note or a photo instead of text?

This handbook assumes plain text messages, which covers the pilot's core scenario. Supporting voice or images would need additional handling (like transcription) before the AI agent ever sees the message — worth scoping as a deliberate future addition rather than assuming it works automatically.

Q: Can this WhatsApp bot and the Claude.ai connector (Handbook 2) both exist at the same time?

Yes — Bharat MCP is the same underlying server either way. Nothing about building this WhatsApp path prevents also pursuing the connector path later; they're two independent doors into the same house.

Q: Who is responsible for what the AI agent says to a customer?

Since Reward360 owns the WhatsApp number, the backend server, and the entire customer relationship on this path, Reward360 carries that responsibility directly — unlike the connector path, where Claude's own interface sits between the customer and Bharat MCP. This is worth keeping in mind when reviewing the provider terms of service mentioned in Section 13.